

National University of Computer and Emerging Sciences



## **TensorSched**

### **Group Members**

Rameen Ramzan ..... 24K-0557

Umama Zubair ..... 24K-0621

### **Supervised by**

**Miss Anaum Hamid**

**FAST School of Computing**

**National University of Computer and Emerging Sciences**

**Karachi, Pakistan**

**May 2026**

## 1. Introduction

TensorSched is a multi-threaded job scheduling simulator developed as part of the Operating Systems course at FAST-NUCES. The project demonstrates core OS concepts including multilevel feedback queue (MLFQ) scheduling, preemptive multithreading, mutex-based synchronization, and semaphore-based resource control. The system ships with a real-time SDL2 graphical user interface that renders live Gantt charts, sparklines, and per-worker status panels.

Project timeline: Spring 2026 semester.

Team: Rameen Ramzan (24K-0557), Umama Zubair (24K-0621).

Supervisor: Miss Anum, FAST School of Computing, NUCES Karachi.

This report covers the following:

- System overview and motivation
- OS concepts applied
- Architecture and design
- Key features
- Technology stack
- Code walk-through with snippets
- Conclusion and future work
- References

## 2. Problem Statement

In real computing environments, multiple processes compete for limited CPU time and memory. Without an intelligent scheduling policy, the system can suffer from starvation of low-priority jobs, poor CPU utilization, and unbounded waiting times. Fixed-priority or simple round-robin schedulers fail to adapt to mixed workloads where jobs have varying burst durations and priority levels.

TensorSched addresses this by implementing a three-level Multilevel Feedback Queue (MLFQ) with aging-based promotion, dynamic preemption via atomic Compare-And-Swap (CAS), and a semaphore-gated worker pool that enforces a hard concurrency limit. The goal

is to maximize CPU utilization, minimize starvation, and fairly balance throughput across priority classes.

### **3. Objectives**

- Simulate concurrent job submission and execution using real POSIX threads.
- Implement MLFQ scheduling with three priority levels, each with a distinct time quantum.
- Apply priority-queue ordering within each MLFQ level so CRITICAL jobs run before HIGH, MEDIUM, and LOW jobs.
- Prevent starvation through an aging mechanism that promotes long-waiting jobs to higher MLFQ levels.
- Enforce a configurable maximum worker concurrency using POSIX semaphores.
- Protect all shared data structures (job queue, log ring buffer, Gantt ring buffer, active-job array) with mutex locks.
- Deliver a real-time SDL2 dashboard showing live Gantt charts, MLFQ depth indicators, resource gauges, and a scrollable event log.
- Provide cross-platform support (Linux/macOS via pthreads; Windows via Win32 shim headers).

## **4. Operating System Concepts Applied**

### **4.1 Multilevel Feedback Queue (MLFQ) Scheduling**

MLFQ is a well-known scheduling algorithm that dynamically adjusts job priority based on runtime behaviour. TensorSched implements three MLFQ levels with time quanta of 2, 4, and 8 ticks respectively. All new jobs enter Level 0 (highest priority). If a job exhausts its quantum without completing, it is demoted to the next level and receives a larger quantum. This automatically identifies CPU-bound jobs and gradually reduces their priority, while short interactive jobs finish quickly at Level 0.

### **4.2 Priority-Queue Ordering**

Within each MLFQ level, jobs are stored in a priority-ordered linked list. Four priority classes are supported: LOW (1), MEDIUM (2), HIGH (3), and CRITICAL (4). When two

jobs share the same priority, FIFO ordering by `submit_time` is used, which provides round-robin fairness at equal priority.

#### **4.3 Preemption with Atomic CAS**

The dispatcher thread periodically calls `checkPreemptions()`, which scans active worker slots. If a higher-priority or higher-level job arrives while a lower-priority job is executing, the running job's status is transitioned from `RUNNING` to `PREEMPTED` using a `compare_exchange_strong` atomic operation. This lock-free CAS ensures that only one thread can successfully claim the preemption, avoiding race conditions without holding the `active-jobs` mutex for extended periods.

#### **4.4 Aging (Starvation Prevention)**

A dedicated aging thread wakes every second and calls `queue_.runAging()`. Any job that has been waiting in Level 1 or Level 2 for more than `AGING_THRESHOLD` (15) ticks is promoted one level upward. Its aging counter and ticks-this-slice counters are reset so it starts fresh at the higher level. This guarantees that no job waits indefinitely regardless of the workload composition.

#### **4.5 Mutex Locks**

Four independent POSIX mutexes protect separate shared regions: `jobs_mutex_` guards the `all_jobs_` array and count; `active_mutex_` guards the `active_jobs_[]` slot array; `gantt_mutex_` guards the circular Gantt record buffer; `log_mutex_` guards the circular log entry buffer. Fine-grained locking reduces contention and allows the GUI rendering thread to read snapshots concurrently with worker threads writing new records.

#### **4.6 Semaphore-Based Concurrency Control**

`ResourceManager::acquire()` calls `sem_wait()` on a POSIX semaphore initialized to `MAX_CONCURRENT` (4). This blocks any new worker from starting execution if all four slots are occupied, without busy-waiting. When a job finishes or fails, `release()` calls `sem_post()` to wake the next waiting worker. This models the real OS concept of bounded parallelism on a fixed number of CPU cores.

#### **4.7 Detached vs. Joinable Threads**

Worker threads are spawned as detached (`PTHREAD_CREATE_DETACHED`) so they clean up automatically upon completion. The long-lived dispatcher and aging threads are joinable and are explicitly joined in `Scheduler::stop()` to ensure a clean shutdown sequence.

## 5. System Architecture

The system is divided into five main components that interact through the Scheduler central coordinator.

| Component          | Responsibility                                                                                                                               |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| job.h              | Job data structure: atomic state, MLFQ metadata, timing fields, helper methods.                                                              |
| job_queue.h        | SingleLevelQueue (mutex-protected linked list) and MLFQJobQueue (three-level wrapper with aging and snapshot APIs).                          |
| resource_manager.h | Tracks CPU%, memory MB, and worker slots. Semaphore gates concurrency. WorkerSlot array exposes live status to GUI.                          |
| scheduler.h/.cpp   | Owens the MLFQ queue and resource manager. Runs dispatcher and aging pthreads. Exposes submit, preempt, log, and Gantt snapshot APIs.        |
| main.cpp           | SDL2 event loop. Renders header, resource panel, MLFQ panel, worker slots, job table, Gantt strip, stats dashboard, and log panel at 60 FPS. |

### Thread Model

The scheduler runs three categories of threads simultaneously:

- Dispatcher thread — main scheduling loop, pops jobs from the MLFQ, spawns detached worker threads, and calls checkPreemptions().
- Aging thread — sleeps 1 second, promotes starved jobs, updates stats.
- Worker threads (up to MAX\_CONCURRENT=4) — each acquires resources, simulates job execution tick-by-tick, emits Gantt records, and requeues or completes the job.
- SDL2 main thread — renders the GUI at 60 FPS, reads snapshots from ring buffers without holding locks longer than a snapshot copy.

## 6. Key Features

## 6.1 Real-Time MLFQ Visualization

The left sidebar displays three horizontal bar graphs showing the number of jobs queued at each MLFQ level, updated every 350 ms. Colour coding — teal for L0, amber for L1, coral for L2 — provides instant visual feedback on queue depth and system saturation.

## 6.2 Live Gantt Chart

The Gantt panel renders a scrollable strip of coloured blocks, one block per scheduler tick, showing which job ran on which worker at each moment. Each block is coloured by MLFQ level. This provides a ground-truth view of the scheduling decisions that would otherwise require offline trace analysis.

## 6.3 Sparklines for CPU and Memory

Rolling 120-sample sparkline graphs are displayed for CPU utilization and memory usage. These time-series mini-charts reveal temporal patterns such as resource bursts when a batch of CRITICAL jobs is submitted.

## 6.4 Job Submission Form and Presets

Users can submit jobs via an in-GUI form specifying name, CPU%, memory MB, burst ticks, and priority. Six one-click presets (Critical Burst, High CPU, Memory Hog, Bulk Low, Mixed Batch, Stress Test) allow rapid workload generation for demonstrations.

## 6.5 Statistics Dashboard

A dedicated tab shows average wait time, average turnaround, throughput (jobs/sec), total submitted/completed/failed/preempted/promoted counts, and a CPU utilization progress bar. Statistics are recalculated after every job completion.

## 6.6 Cross-Platform Shims

`pthread_win.h` and `semaphore_win.h` provide inline Win32 implementations of the POSIX threading API using `CRITICAL_SECTION`, `CONDITION_VARIABLE`, `HANDLE`-based threads, and `CreateSemaphore`. `platform.h` selects the correct headers at compile time, making the entire scheduler layer portable without any source changes.

# 7. Technology Stack

| Technology | Purpose                                                                                                                    |
|------------|----------------------------------------------------------------------------------------------------------------------------|
| C++17      | Core language — <code>atomic&lt;&gt;</code> , <code>std::function</code> , <code>std::chrono</code> , structured bindings. |

|                          |                                                                                                                                                              |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| POSIX Threads (pthreads) | Thread creation, mutex locks, and condition variables on Linux/macOS.                                                                                        |
| POSIX Semaphores         | Bounded concurrency control via <code>sem_wait/sem_post</code> .                                                                                             |
| Win32 Thread API         | <code>pthread_win.h</code> shim for Windows compatibility ( <code>_beginthreadex</code> , <code>CRITICAL_SECTION</code> , <code>CONDITION_VARIABLE</code> ). |
| SDL2                     | Cross-platform window creation, hardware-accelerated 2D rendering, event handling.                                                                           |
| SDL2_ttf                 | TrueType font rendering for all GUI text using <code>TTF_RenderUTF8_Blended</code> .                                                                         |
| GNU Make                 | Build system — single Makefile invokes <code>g++</code> with <code>sdl2-config</code> flags and <code>-lpthread</code> .                                     |

## 8. Code Walk-Through

### 8.1 Job Data Structure (`job.h`)

Every job is represented by a non-copyable `Job` struct. Mutable scheduling fields use `std::atomic<>` to allow lock-free reads from the GUI thread while worker threads update them:

```
struct Job {
    int id;
    char name[64];
    JobPriority priority;
    int cpu_required, memory_required, burst_time;
    std::atomic<JobStatus> status;
    std::atomic<int> elapsed_ticks{0};
    std::atomic<int> mlfq_level{0};
    std::atomic<int> ticks_this_slice{0};
    std::atomic<int> aging_ticks{0};
    bool quantumExpired() const { ... }
    int remainingTicks() const { ... }
};
```

### 8.2 MLFQ Push with Priority Ordering (`job_queue.h`)

`SingleLevelQueue::push()` inserts a job in priority order. Jobs with higher `priorityValue()` are inserted before lower-priority ones; ties break on `submit_time` (FIFO):

```

void push(Job* job) {
    pthread_mutex_lock(&mutex_);
    // walk list to find insertion point
    while (cur) {
        if (job->priorityValue() > cur->job->priorityValue()) break;
        if (job->priorityValue() == cur->job->priorityValue()
            && job->submit_time < cur->job->submit_time) break;
        prev = cur; cur = cur->next;
    }
    // splice node in
    pthread_mutex_unlock(&mutex_);
}

```

### 8.3 CAS-Based Preemption (scheduler.cpp)

`requestPreemption()` uses `compare_exchange_strong` to atomically transition a job from `RUNNING` to `PREEMPTED`. Only the thread that wins the CAS records the preemption event — all others see the job already in `PREEMPTED` state and silently return `false`:

```

bool Scheduler::requestPreemption(Job* target, const char* reason) {
    JobStatus expected = JobStatus::RUNNING;
    if (target->status.compare_exchange_strong(
        expected, JobStatus::PREEMPTED)) {
        stats_.total_preemptions++;
        target->preemption_count++;
        addLog(/* preemption log entry */);
        return true;
    }
    return false; // already preempted by another thread
}

```

### 8.4 Semaphore-Gated Resource Acquisition (resource\_manager.h)

`acquire()` blocks on `sem_wait()` until a worker slot is free, then checks that CPU and memory budgets can accommodate the job. If the resource check fails, `sem_post()` releases the slot immediately so other jobs can proceed:

```

bool acquire(int cpu, int mem) {
    sem_wait(&slots_); // block until a slot opens
    pthread_mutex_lock(&mutex_);
    bool ok = (used_cpu_ + cpu <= TOTAL_CPU) &&
              (used_memory_ + mem <= TOTAL_MEMORY);
    if (ok) { used_cpu_ += cpu; used_memory_ += mem; }
}

```



```

        pthread_mutex_unlock(&mutex_);
        if (!ok) sem_post(&slots_); // give slot back
        return ok;
    }
}

```

## 8.5 Aging Promotion (job\_queue.h)

`drainAged()` removes jobs whose `aging_ticks` have reached `AGING_THRESHOLD` from a lower level and returns them. `runAging()` then re-inserts each promoted job one level higher with a fresh slice counter:

```

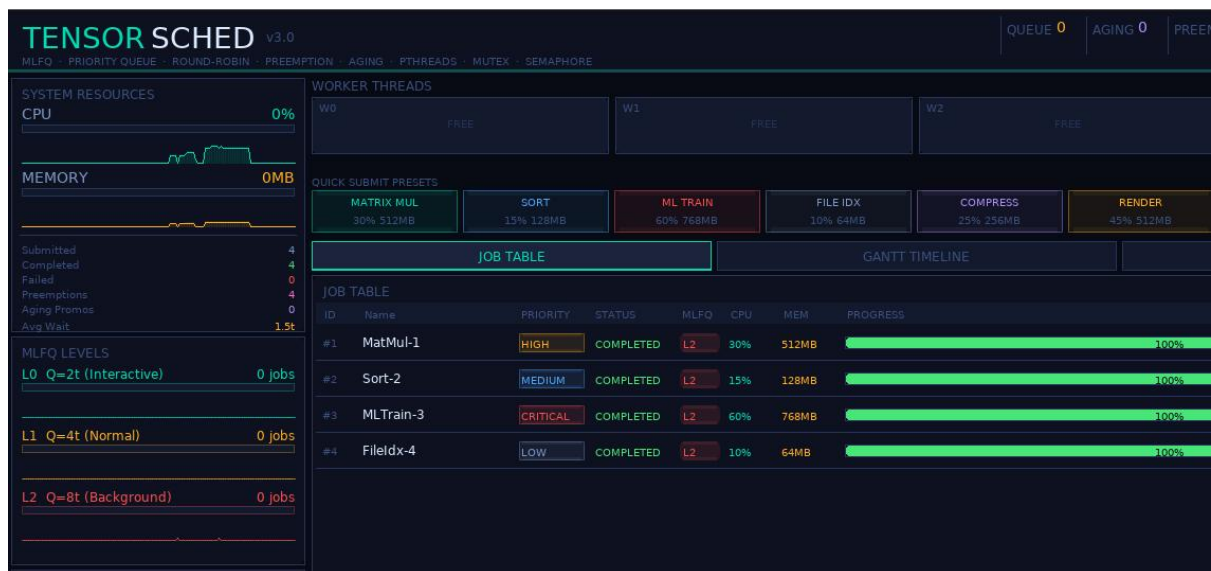
int MLFQJobQueue::runAging() {
    for (int lvl = 1; lvl < MLFQ_LEVELS; lvl++) {
        int count = levels_[lvl].drainAged(aged, AGED_BUF);
        for (int i = 0; i < count; i++) {
            aged[i]->mlfq_level = lvl - 1;
            aged[i]->aging_ticks = 0;
            aged[i]->ticks_this_slice = 0;
            levels_[lvl - 1].push(aged[i]);
        }
    }
}

```

## 9. Conclusion

TensorSched successfully demonstrates the key scheduling and synchronization concepts taught in an Operating Systems course. The combination of MLFQ scheduling, preemption via atomic CAS, aging-based starvation prevention, semaphore-gated concurrency, and fine-grained mutex locking produces a system that is both theoretically sound and practically robust. The SDL2 real-time dashboard elevates the project beyond a headless simulator by making scheduling decisions immediately visible, which aids both debugging and demonstration.

All five major OS primitives covered in the course — threads, mutexes, semaphores, atomic operations, and scheduling algorithms — are present and exercised under realistic concurrent load. The cross-platform shim layer further demonstrates systems programming depth by re-implementing POSIX APIs on top of the Win32 API without modifying any scheduler source code.



**Future Work**

- Add a timed workload file so users can replay a scripted sequence of job submissions for repeatable benchmarks.
- Implement a proper thread pool to eliminate the detached-thread lifetime issue identified during review.
- Extend the resource model to include GPU units and I/O bandwidth, simulating heterogeneous computing environments.
- Add a feedback-driven auto-tuning mode that adjusts AGING\_THRESHOLD and MLFQ\_QUANTUM values based on observed average wait times.
- Export Gantt and statistics data to CSV for offline analysis.

## **10. References**

[1] A. Silberschatz, P. B. Galvin, and G. Gagne, Operating System Concepts, 10th ed. Hoboken, NJ: Wiley, 2018.